

Interfaces

The traditional OOP languages use rigid, nested **"Inheritance"** trees to achieve **polymorphism**. Go completely throws away inheritance and uses **Interfaces** instead.

1. What is an Interface?

An Interface is simply a **contract**. It is a custom type that lists a specific set of method signatures (the **name**, **inputs**, and **outputs**), but it contains absolutely **zero implementation code**.

- **The Concept:** You create an interface to express behavior.
- **Example:** You create an interface called **Shape2D**. You define that to be considered a Shape2D, an object must have two specific methods: **Area()** and **Perimeter()**.

2. "Implicit" Satisfaction (The Magic of Go)

In languages like Java or TypeScript, if you want a **Triangle** class to use an interface, you have to explicitly announce it to the compiler (e.g., **class Triangle implements Shape2D**).

In Go, interfaces are satisfied implicitly.

- You do not need an **implements** keyword.
- If you create a **Triangle struct**, and you attach an **Area()** method and a **Perimeter()** method to it, the Go compiler automatically recognizes that Triangle satisfies the **Shape2D interface**.

Note: Your Triangle can have **50** other custom methods and **20** unique data fields. The interface doesn't care. As long as it has the required methods, it fits the contract!

3. The Syntax in Action

Here is how you define the contract, and how a struct silently fulfills it:

Step 1: Define the Interface

```
type Shape2D interface {  
    Area() float64  
    Perimeter() float64  
}
```

Step 2: Create a Struct (The Data)

```
type Triangle struct {  
    base    float64  
    height  float64  
    sideA   float64  
    sideB   float64  
    sideC   float64  
}
```

Step 3: Fulfill the Contract (The Methods)

```
// As soon as you write these two methods, Triangle automatically becomes a Shape2D!
func (t Triangle) Area() float64 {
    return 0.5 * t.base * t.height
}

func (t Triangle) Perimeter() float64 {
    return t.sideA + t.sideB + t.sideC
}
```

Expert Takeaway:

Because Go interfaces are implicit, they create incredibly decoupled and flexible code, which programmers often call "**Duck Typing**" ("If it walks like a duck and quacks like a duck, it's a duck").

If you are writing a custom web framework, you can create a **Writer** interface that requires a **Write()** method. Now, your framework can accept a Network Connection, a File, or an HTTP Response object, because all of those built-in Go types happen to have a **Write()** method attached to them! You don't have to rewrite any underlying source code to make them compatible; they just work together seamlessly out of the box.